

프로젝트 명세서

블록체인 프로젝트

“스마트 컨트랙트 기반 예측 시스템”

Sub PJT I

목차

1. 프로젝트 개요	3
2. 프로젝트 목표	11
3. 필수 지식 학습	13
1) 솔리디티 기본 문법	13
2) ERC-20 토큰 기본 개념	14
4. 과제 명세	16
1) 과제 1: 테스트용 ERC-20 구현	16
2) 과제 2: 예측 컨트랙트 구현	26
3) 과제 3: 정산 컨트랙트 구현	35

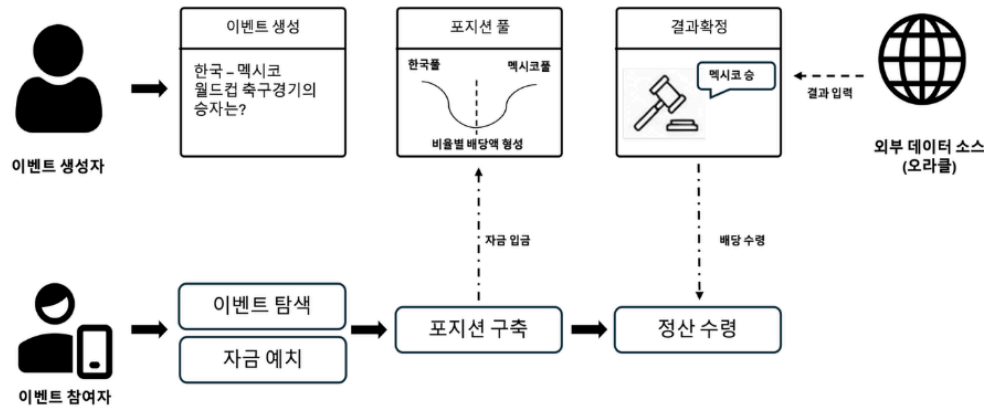
1. 프로젝트 개요

이번 프로젝트는 블록체인 기반 예측 이벤트 플랫폼을 구축하는 것을 목적으로 합니다. 예측 이벤트 플랫폼은 특정 주제에 대해 사용자가 결과를 예측하고, 자신이 선택한 결과에 토큰 양을 정해서 참여한 뒤, 실제 결과가 확정되면 스마트 컨트랙트의 규칙에 따라 정산을 받는 서비스입니다. 일반적인 웹 서비스에서도 이벤트 생성, 참여자 관리, 결과 입력, 정산 내역 조회와 같은 기능은 구현할 수 있습니다. 그러나 예측 참여 금액의 보관, 결과 확정 이후의 정산 기준, 수수료와 추가 배당금의 처리처럼 이해관계가 직접 연결되는 영역은 중앙 서버의 내부 로직만으로 처리할 경우 운영자의 임의 변경 가능성이나 정산 과정의 불투명성이 문제로 남을 수 있습니다.

블록체인은 이러한 문제를 완화하기 위한 좋은 실습 주제가 됩니다. 스마트 컨트랙트는 한 번 배포되면 정해진 규칙에 따라 실행되고, 예측 참여와 정산에 필요한 주요 상태가 블록체인 위에 기록됩니다. 사용자는 자신이 어떤 이벤트에 얼마를 참여했는지, 어떤 결과가 확정되었는지, 정산 가능한 금액이 얼마인지 컨트랙트 상태를 통해 확인할 수 있습니다. 운영자는 이벤트를 생성하고 결과를 확정하지만, 참여 금액의 이동과 정산 금액 계산은 컨트랙트에 작성된 규칙을 따릅니다. 따라서 이 프로젝트는 단순히 화면과 API를 만드는 것이 아니라, 신뢰가 필요한 서비스 로직을 온체인 로직과 오프체인 시스템이 어떻게 나누어 처리하는지 학습하는 데 초점을 둡니다.

예측 서비스는 스포츠 경기, 교육 과정 이벤트, 설문형 의사결정, 커뮤니티 챌린지 등 다양한 형태로 확장할 수 있습니다. 예를 들어 "한국 - 멕시코 월드컵 축구경기 예측"과 같은 이벤트를 생성하면 사용자는 승리 팀 또는 경기 결과를 선택하고 토큰을 전송합니다. 이벤트 종료 후 관리자가 실제 결과를 입력하면 시스템은 적중자와 비적중자를 구분하고, 컨트랙트에 저장된 풀(pool) 정보와 수수료율을 기준으로 정산 금액을 계산합니다. 추가 배당금이 설정된 이벤트라면 해당 금액은 적중자의 보상 풀에 포함될 수 있으며, 정답 선택지를 선택한 사용자가 없는 경우에는 환불 모드와 같은 별도 정책을 적용할 수 있습니다.

사용자 예측 참여, 토큰 예치, 결과 확정, 정산 데이터 생성, 배당 전송까지 이어지는 순서도를 넣으면 프로젝트의 핵심 흐름을 빠르게 이해할 수 있습니다.



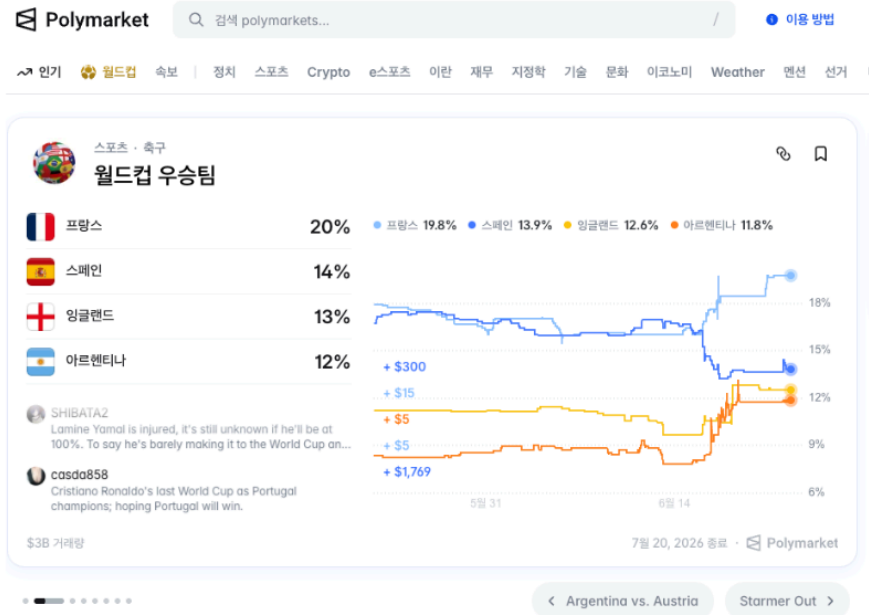
<그림 1> 예측 이벤트의 전체 흐름도

현재 가장 널리 알려진 블록체인 기반 예측 시장 사례로는 폴리마켓 (<https://polymarket.com>)을 들 수 있습니다. 폴리마켓은 정치, 스포츠, 경제 지표, 국제 이슈, 날씨, 문화 이벤트 등 현실 세계의 다양한 사건을 예측 대상으로 삼고, 사용자가 특정 결과의 가능성을 나타내는 포지션을 거래할 수 있도록 만든 예측 시장 플랫폼입니다. 일반적인 투표나 설문이 사람들의 의견을 묻는 방식이라면, 예측 시장은 실제 자산을 걸고 거래한다는 점에서 참여자의 판단에 경제적 인센티브가 연결됩니다. 이 때문에 시장 가격은 특정 결과가 발생할 가능성에 대한 집단적 기대를 나타내는 지표처럼 해석되기도 합니다.

폴리마켓과 같은 서비스의 시스템 구조를 살펴보는 것이 도움이 되는 이유는 예측 시장이 단순한 게시판형 투표 서비스가 아니라, 블록체인과 토큰, 지갑, 시장 가격, 정산 규칙이 결합된 DApp 형태로 구현될 수 있음을 보여주기 때문입니다. 사용자는 암호화폐 지갑을 이용해 자산을 예치하고, 이벤트 결과에 따라 가치가 달라지는 포지션을 거래합니다. 이벤트가 종료되면 정해진 기준에 따라 결과가 확정되고, 승리한 포지션을 보유한 사용자는 정산을 받습니다. 이 흐름은 본 프로젝트에서 다루는 예측 이벤트 생성, 예측 참여, 결과 확정, 정산 처리와 구조적으로 유사합니다.

또한 폴리마켓은 예측 시장의 가능성과 한계를 함께 보여주는 사례입니다. 2025년에는 뉴욕증권거래소의 모회사인 Intercontinental Exchange가 폴리마켓에 대규모 투자를 진행하면서 예측 시장이 전통 금융권에서도 중요한 데이터와 시장 영역으로 주목받고 있음을 보여주었습니다. 동시에 국가별 규제, 내부 정보 이용 가능성, 결과 판정 기준의 모호성, 일반 사용자 보호 문제와 같은 논란도 계속 제기되고 있습니다. 따라서 이 프로젝트에서는 상업적 예측 시장을 그대로 모방하기보다, 교육 환경에 맞게 제한된 참여

대상, 명확한 이벤트 운영자, 단순한 정산 규칙, 투명한 스마트 컨트랙트 상태를 중심으로 예측 시스템을 설계합니다.



<그림 2> 폴리마켓의 메인 화면(출처 : Polymarket)

폴리마켓의 "시장 생성 - 포지션 거래 - 결과 판정 - 정산" 흐름과 본 프로젝트의 "이벤트 생성 - 예측 참여 - 결과 확정 - 정산 전송" 흐름을 나란히 비교하면 구현 범위와 학습 목표를 설명하기 좋습니다.

예측 이벤트에서 가장 중요한 요소는 "누가 어떤 선택지에 얼마를 참여했는가"와 "결과 확정 이후 얼마를 돌려받는가"입니다. 이 두 정보는 참여자의 자산과 직접 연결됩니다. 기존 중앙화 시스템에서는 모든 참여 내역과 정산 계산이 서버와 데이터베이스에 저장됩니다. 사용자는 서버가 제공하는 화면을 통해 결과를 확인하지만, 실제 계산 과정이 변경되지 않았는지 또는 운영자가 임의로 특정 참여자를 누락하지 않았는지 직접 검증하기 어렵습니다.

스마트 컨트랙트를 활용하면 이러한 핵심 규칙을 공개된 코드와 상태로 표현할 수 있습니다. 예측 참여 함수는 동일 사용자의 중복 참여를 막고, 이벤트별 선택지와 선택지별 전송 금액을 저장합니다. 결과 확정 함수는 운영 권한을 가진 주소만 호출할 수 있도록 제한하고, 확정된 결과를 기준으로 정산 가능한 금액을 계산합니다. 정산 함수는 수신

자 주소와 보상 금액을 기준으로 토큰 전송을 실행하며, 이미 정산된 사용자가 중복으로 보상을 받지 못하도록 상태를 기록합니다.

다만 블록체인만으로 서비스 전체를 만들지는 않습니다. 사용자가 보기 편한 이벤트 목록, 검색과 필터, 관리자 화면, 알림, 마이페이지 거래 내역 같은 기능은 여전히 웹 프론트엔드와 백엔드가 담당합니다. 이 프로젝트에서 여러분은 어떤 로직을 스마트 컨트랙트에 두어야 하고, 어떤 로직을 백엔드와 데이터베이스에 두어야 하는지 구분하게 됩니다. 예측 참여 금액과 정산처럼 신뢰와 검증이 필요한 영역은 컨트랙트가 담당하고, 사용자 경험과 운영 편의성을 위한 조회, 캐싱, 알림, 관리 기능은 오프체인 시스템이 보조합니다.

이러한 구조는 실제 DApp 개발에서 자주 사용되는 방식입니다. 모든 데이터를 온체인에 저장하면 비용과 성능 문제가 발생하고, 모든 것을 오프체인에 저장하면 블록체인을 사용하는 의미가 약해집니다. 따라서 본 프로젝트는 "온체인 상태와 오프체인 애플리케이션의 역할 분리"를 자연스럽게 경험할 수 있도록 설계합니다.

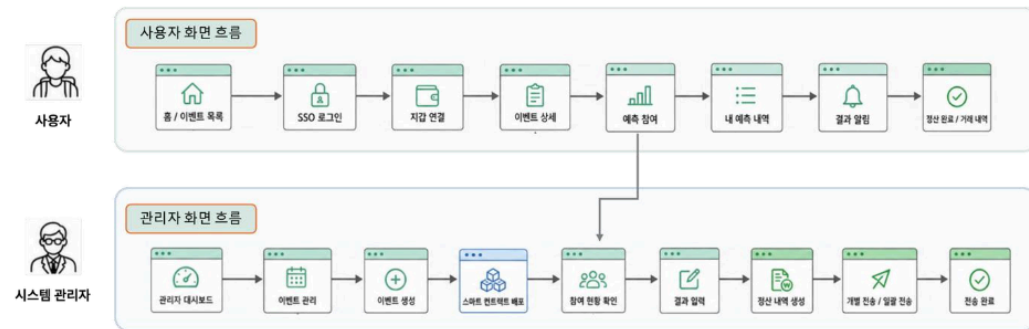
서비스의 기본 사용자는 사용자와 관리자입니다. 사용자는 로그인을 한 후, 예측에 참여하기 위해 메타마스크 지갑을 연결합니다. 시스템은 로그인한 사용자의 정보에 따라 이벤트의 참여 가능 조건을 판단하는 데 사용됩니다. 사용자는 자신이 참여 가능한 이벤트를 확인하고, 이벤트 상세 화면에서 선택지를 고른 뒤 토큰을 전송합니다.

관리자는 관리자 대시보드에서 예측 이벤트를 생성합니다. 이벤트 생성 시 제목, 설명, 이미지, 카테고리, 시작 시각, 종료 시각, 참여 가능 기수와 지역/반 조건, 예측 옵션, 수수료율, 추가 배당금 등을 입력합니다. 이벤트가 생성되면 백엔드는 데이터베이스에 이벤트 정보를 저장하고, 동시에 팩토리 컨트랙트를 통해 개별 예측 이벤트 컨트랙트를 배포합니다. 이때 각 이벤트는 독립된 컨트랙트 주소를 가지며, 해당 컨트랙트가 참여 금액과 결과 확정, 정산 정보를 관리합니다.

이벤트가 진행되는 동안 사용자는 한 이벤트에 한 번만 참여할 수 있습니다. 컨트랙트는 동일 주소의 중복 참여를 막고, 프론트엔드도 사용자가 이미 참여한 이벤트에서는 추가 참여를 할 수 없도록 제어합니다. 참여가 완료되면 백엔드는 참여 내역을 데이터베이스에 저장하고, 화면에는 최근 예측 내역과 참여자 목록이 표시됩니다. 이때 온체인 트랜잭션 해시와 사용 지갑 주소를 함께 저장하면 이후 마이페이지 거래 내역에서 사용자가 자신의 참여 기록을 확인할 수 있습니다.

이벤트 종료 후 관리자는 결과를 입력합니다. 결과가 확정되면 컨트랙트는 정답 선택지를 기록하고, 백엔드는 컨트랙트 상태를 바탕으로 정산 대상자와 정산 금액을 데이터베이스에

이스에 생성합니다. 이후 관리자는 정산 내역 관리 화면에서 전송 대기 상태의 정산 건을 확인하고, 개별 전송 또는 일괄 전송을 실행할 수 있습니다. 전송이 완료되면 정산 상태는 전송 완료로 변경되며, 사용자에게 해당 전송 완료 알림이 전달됩니다.



<그림 3> 사용자와 관리자 역할별 화면 흐름도

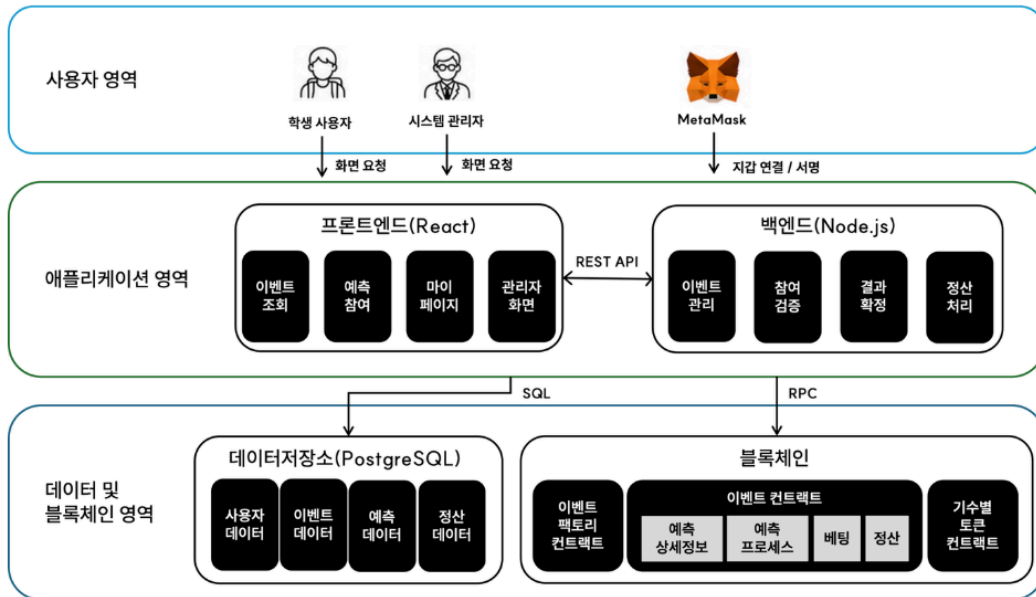
사용자의 "로그인 - 지갑 연결 - 이벤트 조회 - 예측 참여 - 마이페이지 확인" 흐름과 관리자의 "이벤트 생성 - 결과 확정 - 정산 전송 - 참여 현황 확인" 흐름을 나란히 배치하면 서비스 역할이 명확해집니다.

프로젝트는 프론트엔드, 백엔드, 스마트 컨트랙트 세 영역으로 구성됩니다. 프론트엔드는 React 와 TypeScript 기반의 웹 애플리케이션이며, 사용자와 관리자가 사용하는 화면을 제공합니다. 사용자 화면에는 홈 이벤트 목록, 이벤트 상세, 예측 참여, 마이페이지, 알림 패널이 포함됩니다. 관리자 화면에는 이벤트 생성과 관리, 이벤트 상세 편집, 결과 확정, 참여 현황, 정산 내역 관리, 기수 관리, 관리자 계정 관리가 포함됩니다.

백엔드는 Node.js 와 Express 기반 API 서버입니다. 백엔드는 사용자 정보 동기화, 이벤트 데이터 조회와 저장, 예측 참여 내역 저장, 결과 확정 처리, 정산 데이터 생성, 알림 생성, 공통 코드 조회, 관리자 기능을 담당합니다. 데이터베이스는 PostgreSQL 을 사용하며, 이벤트 정보, 사용자 정보, 기수 정보, 참여 내역, 정산 내역, 알림 정보를 저장합니다. 백엔드는 블록체인 연동 모듈을 통해 **PredictionEventFactory** 컨트랙트와 개별 **PredictionEvent** 컨트랙트를 호출합니다.

스마트 컨트랙트는 솔리디티(Solidity)로 작성됩니다. 핵심 컨트랙트는 **PredictionEventFactory** 와 **PredictionEvent** 입니다. **PredictionEventFactory** 는 새로운 예측 이벤트 컨트랙트를 생성하고 이벤트 ID 와 컨트랙트 주소를 연결합니다. **PredictionEvent** 는 개별 예측 이벤트 하나의 상태를 담당하며, 예측 참여, 선택지별 풀 집계, 추가 배당금 입금 확인, 결과 확정, 정산 금

액 계산, 보상 전송, 수수료와 잔여 보상 회수 기능을 포함합니다. 참여와 정산에 사용되는 토큰은 ERC-20 토큰을 기준으로 합니다.



<그림 4> 전체 시스템 구성도

분류	기술 스택	버전	요약
Blockchain Environment	Besu	21.10.2	블록체인 환경은 싸피에서 직접 제공합니다.
Frontend	MetaMask Profer API		지갑 연결, 토큰 관리, 예측 참여
	React	17.19.1.0	Javascript API for UI
	TypeScript	5.8.3	프론트엔드 정적타입
Backend	Node.js	24.1.0	Javascript Runtime
	Express	4.21.2	API Server
	PostgreSQL	16	RDBMS

Smart Contract	Solidity	0.8.24	이더리움 스마트 컨트랙트 작성 언어
----------------	----------	--------	---------------------

이 프로젝트에서 중요한 점은 특정 기술을 단순히 나열하는 것이 아니라, 각 기술이 맡는 책임을 이해하는 것입니다. React 는 사용자가 이벤트를 쉽게 탐색하고 참여할 수 있도록 화면을 구성합니다. Express 서버는 화면과 데이터베이스, 블록체인 사이에서 요청을 조율합니다. PostgreSQL 은 조회 성능과 운영 편의성을 위해 필요한 데이터를 저장합니다. 솔리디티 컨트랙트는 예측 참여와 정산처럼 신뢰가 필요한 핵심 규칙을 실행합니다. 메타마스크와 ethers 는 사용자의 지갑과 블록체인 네트워크를 연결하는 역할을 합니다.

본 프로젝트는 총 3 개의 Sub PJT 로 구성됩니다. 각 Sub PJT 는 독립된 학습 목표를 가지며, 앞 단계의 결과물이 다음 단계의 기반이 되도록 설계합니다. Sub PJT 가 뒤로 갈수록 단순한 컨트랙트 실습에서 팩토리 기반 이벤트 생성, 운영 화면과 정산 시스템 연동으로 범위가 확장됩니다.

■ Sub PJT 1: 예측 컨트랙트 기초와 토큰 실습

Sub PJT 1에서는 완성 프로젝트보다 단순한 형태의 예측 컨트랙트를 작성합니다. 여러분은 솔리디티 문법, ERC-20 토큰 전송 방식, 예측 선택지 저장, 사용자 예측 참여, 결과 확정, 정산 계산의 기본 개념을 학습합니다. 이 단계의 목적은 복잡한 웹 시스템을 구현하는 것이 아니라, 예측 서비스의 핵심 규칙이 스마트 컨트랙트에서 어떤 상태 변수와 함수로 표현되는지 이해하는 것입니다. 테스트 코드를 통해 사용자가 예측에 참여하고, 결과가 확정된 뒤 정산 금액이 계산되는 흐름을 검증합니다.

■ Sub PJT 2: 팩토리 기반 예측 컨트랙트

Sub PJT 2에서는 여러 개의 예측 이벤트를 생성할 수 있도록 팩토리 구조를 도입합니다. 하나의 예측 이벤트만 다루는 컨트랙트에서 벗어나, 이벤트마다 독립적인 **PredictionEvent** 컨트랙트를 배포하고 팩토리가 생성된 컨트랙트 주소를 관리하는 구조를 구현합니다. 이 단계에서는 이벤트 생성과 조회 기능이 구체화됩니다. 또한 백엔드나 프론트엔드가 컨트랙트 주소와 ABI 를 이용해 이벤트별 컨트랙트를 호출하는 방식도 함께 이해합니다.

■ Sub PJT 3: 예측 이벤트 운영과 예측 시스템의 정산

Sub PJT 3에서는 완성형 예측 시스템에 가까운 운영 기능을 구현합니다. 사용자는 화면에서 예측에 참여하고, 결과 확정 이후에는 정산 대상자와 정산 금액이 생성됩니다. 관리자는 정산 내역 관리 화면에서 전송 대기 상태의 정산을 확인하고 보상 전송을 실행합니다. 추가 배당금, 환불 모드, 잔여 토큰 회수 같은 정책은 선택 과업으로 확장할 수 있습니다.

이와 같은 3 단계 구성은 여러분이 스마트 컨트랙트의 기본 원리를 먼저 익힌 뒤, 여러 컨트랙트를 생성하는 구조와 실제 웹 서비스 운영 흐름까지 점진적으로 경험하도록 돕습니다. 최종적으로 여러분은 블록체인 기반 예측 시스템을 통해 단순한 토큰 전송을 넘어, 현실적인 DApp에서 발생하는 이벤트 운영, 사용자 참여, 결과 확정, 정산, 알림, 관리자 도구의 연결 방식을 종합적으로 학습하게 됩니다.

2. 프로젝트 목표

Sub PJT 1을 완료하면 여러분은 솔리디티 기반 스마트 컨트랙트의 기본 구조를 이해하고, ERC-20 토큰을 활용한 컨트랙트 호출 흐름을 설명할 수 있어야 합니다. 또한 예측 참여, 결과 확정, 정산 계산처럼 예측 시스템의 핵심 규칙을 온체인 로직으로 표현할 수 있어야 합니다.

첫 번째 목표는 솔리디티 문법과 스마트 컨트랙트 상태 관리 방식에 익숙해지는 것입니다. 예측 이벤트 컨트랙트는 선택지, 참여자, 참여 금액, 결과 확정 여부, 정답 선택지와 같은 상태를 저장해야 합니다. 여러분은 이러한 정보를 상태 변수, **mapping**, **struct**, 배열로 표현하고, 각 상태가 함수 호출에 따라 어떻게 변경되는지 확인합니다.

두 번째 목표는 ERC-20 토큰을 이용한 예측 참여 흐름을 이해하는 것입니다. 사용자가 컨트랙트에 토큰을 맡기려면 일반적으로 토큰 컨트랙트에 **approve**를 먼저 호출하고, 예측 컨트랙트가 **transferFrom**을 통해 사용자의 토큰을 가져오는 구조를 사용합니다. Sub PJT 1에서는 이 흐름을 직접 구현하고 테스트함으로써, DApp에서 토큰 전송 권한과 실제 토큰 이동이 분리되는 이유를 이해합니다.

세 번째 목표는 예측 결과 확정과 정산 계산 규칙을 구현하는 것입니다. 예측 이벤트는 결과가 확정되기 전과 확정된 후의 동작이 명확히 달라야 합니다. 결과 확정 전에는 사용자가 예측에 참여할 수 있지만, 결과 확정 후에는 더 이상 참여할 수 없어야 합니다. 결과가 확정되면 컨트랙트는 전체 참여 금액, 정답 선택지의 전송된 금액, 사용자별 참여 금액을 기준으로 적중자의 예상 정산 금액을 계산할 수 있어야 합니다.

네 번째 목표는 테스트 코드를 통해 컨트랙트의 정상 동작을 검증하는 것입니다. 스마트 컨트랙트는 배포 후 수정이 어렵기 때문에, 구현 단계에서 단위 테스트와 시나리오 테스트를 작성하는 습관이 중요합니다. 여러분은 예측 참여, 중복 참여 방지, 결과 확정, 정산 금액 계산이 의도한 대로 동작하는지 테스트 코드로 확인합니다.

구분	구현 내용
----	-------

테스트용 ERC-20 토큰	예측 참여에 사용할 기본 ERC-20 토큰을 준비합니다.
예측 선택지	최소 2 개의 선택지를 저장하고 조회할 수 있도록 구현합니다.
예측 참여	사용자가 특정 선택지에 토큰을 전송하여 참여할 수 있도록 구현합니다.
참여 금액 집계	선택지별 총 참여 금액과 전체 참여 금액을 저장합니다.
결과 확정	운영자만 정답 선택지를 확정할 수 있도록 구현합니다.
정산	적중자가 받을 수 있는 정산 금액을 계산합니다.

3. 필수 지식 학습

Sub PJT 1에서는 예측 시스템의 가장 작은 단위인 단일 예측 컨트랙트를 구현합니다. 이를 위해서는 솔리디티 문법, 스마트 컨트랙트의 상태 관리 방식, ERC-20 토큰 전송 구조, 컨트랙트 테스트 방법을 먼저 이해해야 합니다. 이번 장에서는 Sub PJT 1을 수행하기 전에 반드시 학습해야 할 핵심 지식을 정리합니다.

Sub PJT 1의 목적은 완성형 서비스를 바로 구현하는 것이 아니라, 예측 참여와 정산 계산이 스마트 컨트랙트에서 어떻게 표현되는지 이해하는 것입니다. 따라서 프론트엔드 화면, 백엔드 API, 관리자 운영 화면보다 컨트랙트의 상태 변수와 함수 설계에 집중해야 합니다.

1) 솔리디티 기본 문법

솔리디티는 이더리움 계열 블록체인에서 스마트 컨트랙트를 작성하기 위한 언어입니다. 스마트 컨트랙트는 블록체인에 배포된 뒤 외부 계정 또는 다른 컨트랙트의 호출에 의해 실행됩니다. 일반적인 서버 프로그램과 달리 컨트랙트의 상태는 블록체인에 저장되며, 상태를 변경하는 함수 호출은 트랜잭션으로 처리됩니다.

Sub PJT 1을 수행하기 위해서는 다음 솔리디티 기본 요소를 이해해야 합니다.

구분	학습 내용
상태 변수	컨트랙트에 영구 저장되는 값입니다. 예측 선택지, 참여자 정보, 참여 금액 등을 저장하는 데 사용합니다.
지역 변수	함수 실행 중에만 사용되는 값입니다. 정산 금액 계산이나 입력값 검증에 활용합니다.
mapping	주소별 참여 여부, 사용자별 예측 참여 정보, 선택지별 참여 금액을 저장하는 데 사용합니다.
struct	여러 값을 하나의 자료형처럼 묶어 참여 정보나 선택지 정보를 표현할 때 사용합니다.
array	선택지 목록처럼 순서가 있는 데이터를 저장할 때 사용합니다.

modifier	특정 함수에 반복되는 검증 조건을 적용할 때 사용합니다. 예를 들어 운영자만 호출 가능한 함수에 사용할 수 있습니다.
event	컨트랙트에서 중요한 상태 변화가 발생했음을 외부에 알릴 때 사용합니다.
require	입력값이나 상태 조건을 검증하고, 조건을 만족하지 않으면 실행을 중단합니다.

예측 컨트랙트는 상태 변화가 명확해야 합니다. 사용자가 예측에 참여하면 참여 여부, 선택한 선택지, 참여 금액, 선택지별 총 참여 금액이 변경됩니다. 운영자가 결과를 확정하면 결과 확정 여부와 정답 선택지가 변경됩니다. 이처럼 어떤 함수가 어떤 상태를 변경하는지 명확히 구분할 수 있어야 합니다.

또한 스마트 컨트랙트를 컴파일하고 이를 활용할 수 있는 도구가 필요합니다. 보통 이더리움 계열 스마트 컨트랙트를 개발하는 IDE로는 별도의 개발 환경 구축 없이도 간편하게 사용할 수 있는 Remix가 있습니다. 솔리디티 문법이 어려우신 분들은 CryptoZombie 사이트를 통해 간단한 게임을 제작하며 익힐 수 있습니다.

구분	참고 링크
솔리디티 공식 문서	https://docs.soliditylang.org/en/v0.8.24
솔리디티 기본 자료형	https://docs.soliditylang.org/en/v0.8.24/types.html
다른 컨트랙트 호출하기	https://medium.com/@blockchain101/calling-the-function-of-another-contract-in-solidity-f9edfa921f4c
Remix 사용법	https://remix-ide.readthedocs.io/en/latest/
예제로 배우는 솔리디티 - CryptoZombie	https://cryptozombies.io/

2) ERC-20 토큰 기본 개념

ERC-20은 이더리움 계열 블록체인에서 가장 널리 사용되는 대체 가능 토큰 표준입니다. 대체 가능 토큰이란 같은 단위끼리 서로 같은 가치를 가지는 토큰을 의미합니다. 예측

시스템에서는 사용자가 예측에 참여할 때 ERC-20 토큰을 예측 참여 금액으로 사용합니다.

Sub PJT 1에서는 테스트용 ERC-20 토큰을 준비하고, 사용자가 해당 토큰을 예측 컨트랙트에 맡기는 흐름을 구현합니다. 이때 반드시 이해해야 할 함수는 다음과 같습니다.

함수	역할
<code>balanceOf(address)</code>	특정 주소가 보유한 토큰 잔액을 조회합니다.
<code>transfer(address, uint256)</code>	호출자가 자신의 토큰을 다른 주소로 전송합니다.
<code>approve(address, uint256)</code>	특정 주소가 호출자의 토큰을 대신 사용할 수 있도록 허용합니다.
<code>allowance(address, address)</code>	특정 주소가 대신 사용할 수 있도록 허용된 토큰 수량을 조회합니다.
<code>transferFrom(address, address, uint256)</code>	허용된 범위 안에서 다른 주소의 토큰을 전송합니다.

예측 컨트랙트가 사용자의 토큰을 가져오기 위해서는 사용자가 먼저 토큰 컨트랙트에 **approve**를 호출해야 합니다. **approve**를 통해 예측 컨트랙트 주소에 일정 수량의 토큰 사용 권한을 부여하면, 예측 컨트랙트는 **transferFrom**을 호출하여 사용자의 토큰을 자신의 주소로 이동시킬 수 있습니다. 이 구조는 DApp에서 매우 자주 사용되므로 반드시 이해해야 합니다.

아래의 표준 문서와 ERC-20을 직접 구현한 OpenZeppelin Doc를 활용해 보시길 추천합니다.

	설명 및 링크
ERC-20	ERC-20: Token Standard https://eips.ethereum.org/EIPS/eip-20
OpenZeppelin Doc	OpenZeppelin Docs https://docs.openzeppelin.com/contracts/4.x/erc20

4. 과제 명세

1) 과제 1: 테스트용 ERC-20 구현

예측 시스템에서 사용자는 현금이나 네이티브 코인 대신 프로젝트 환경에서 쓸 수 있는 ERC-20 토큰을 사용하여 예측에 참여합니다. 따라서 예측 컨트랙트를 구현하기 전에 토큰이 어떤 상태를 저장하고, 사용자 사이에서 어떻게 이동하며, 다른 컨트랙트에 사용 권한을 어떻게 위임하는지 이해해야 합니다.

(1) 학습목표

이번 실습을 완료하면 다음 내용을 설명하고 구현할 수 있어야 합니다.

구분	학습 목표
ERC-20 표준	토큰의 특징과 표준 인터페이스를 이해합니다.
초기 발행	최소 단위 관계를 이해하고 최초 토큰 발행을 구현합니다.
토큰 전송	사용자에게 토큰을 전송하는 기능을 구현합니다.
사용 권한 승인	다른 사람(또는 스마트 컨트랙트)에게 토큰 송금 권한을 부여하는 기능을 구현합니다.
대리 전송	승인받은 주소가 토큰을 이동시키는 기능을 구현합니다.
컨트랙트 테스트	구현한 스마트 컨트랙트의 테스트 코드를 작성합니다.

(2) 구현 요구사항

테스트 토큰은 OpenZeppelin의 ERC-20 구현을 상속하여 작성합니다. ERC-20 표준 기능을 처음부터 다시 작성하면 토큰의 핵심 사용 흐름보다 표준 내부 구현에 많은 시간이 필요하고, 권한이나 잔액 처리에서 오류가 발생할 수 있습니다. 검증된 라이브러리를 활용하고 상속된 기능을 직접 호출해 보는 방식으로 실습을 진행합니다.

필수 구현 요구사항은 다음과 같습니다.

번호	요구사항
----	------

1	컨트랙트 이름은 TestToken 으로 작성합니다.
2	OpenZeppelin 의 ERC20 컨트랙트를 상속합니다.
3	토큰 이름과 심벌은 생성자에서 전달받습니다.
4	초기 공급량은 생성자에서 전달받아 배포자 주소에 발행합니다.
5	토큰의 소수점 자릿수는 ERC-20 기본값인 18 자리를 사용합니다.
6	배포 이후 totalSupply 와 배포자의 balanceOf 가 초기 공급량과 일치해야 합니다.
7	배포자는 transfer 를 통해 두 명 이상의 테스트 사용자에게 토큰을 지급할 수 있어야 합니다.
8	사용자는 approve 를 통해 다른 주소에 토큰 사용 권한을 부여할 수 있어야 합니다.
9	승인 받은 주소는 허용 수량 안에서 transferFrom 을 호출할 수 있어야 합니다.
10	잔액 또는 허용 수량을 초과하는 전송은 실패해야 합니다.

(3) 개발 환경 준비

이번 실습은 Windows 10 또는 Windows 11 환경을 기준으로 진행합니다. 명령어는 Windows PowerShell 에서 실행하며, 코드 편집기는 Visual Studio Code 사용을 권장합니다. Solidity 0.8.24 를 기준으로 컨트랙트를 작성하고, 테스트 환경은 Hardhat 과 ethers 를 사용합니다. ERC-20 표준 구현은 OpenZeppelin Contracts 를 활용합니다.

먼저 다음 프로그램을 설치합니다.

프로그램	용도	확인 방법
Node.js LTS	Hardhat 과 테스트 코드 실행	node --version
npm	프로젝트 패키지 설치 및 스크립트 실행	npm --version
Visual Studio Code	솔리디티 및 JavaScript 코드 작성	시작 메뉴에서 실행

다음 명령어를 통해 Node.js LTS 를 설치합니다.

```
PS C:\Users\student> winget install OpenJS.NodeJS.LTS
```

Node.js 를 설치하면 npm 도 함께 설치됩니다. 프로그램 설치가 끝나면 새 PowerShell 창을 열고 다음 명령으로 설치 상태를 확인합니다.

```
PS C:\Users\student> node -version
PS C:\Users\student> npm --version
```

(4) 프로젝트 설정

실습 파일을 저장할 위치로 이동한 뒤 프로젝트 디렉터리를 생성합니다. 다음 예시는 Windows 사용자의 문서 폴더 아래에 프로젝트를 생성합니다.

```
PS C:\Users\student> Set-Location "$HOME\Ssfy"
PS C:\Users\student\Ssfy> New-Item -ItemType Directory -Name "sub-pjt-1"
PS C:\Users\student\Ssfy> Set-Location ".\sub-pjt-1"
PS C:\Users\student\Ssfy\sub-pjt-1> npm init -y
```

프로젝트를 새로 구성하는 경우 다음과 같은 디렉터리 구조를 권장합니다.

```
sub-pjt-1/
├── contracts/
│   └── TestToken.sol
├── test/
│   └── TestToken.test.js
├── scripts/
│   └── deploy-test-token.js
├── hardhat.config.js
└── package.json
```

PowerShell 에서 프로젝트 최상위 디렉터리로 이동한 상태에서 필요한 패키지를 설치합니다. Hardhat 은 로컬 내에 가상의 이더리움 컴파일 및 실행 환경을 제공해주는 이더리움 개발 프레임워크이고, OpenZeppelin 은 안전한 스마트 컨트랙트를 빠르게 개발할 수 있도록 표준 계약과 보안 도구를 제공하는 오픈소스 라이브러리입니다.

```
PS C:\Users\student\Ssfy\sub-pjt-1> npm install --save-dev hardhat
PS C:\Users\student\Ssfy\sub-pjt-1> npm install --save-dev @nomicfoundation/hardhat-toolbox-mocha-ethers
PS C:\Users\student\Ssfy\sub-pjt-1> npm install @openzeppelin/contracts
```

설치가 완료되면 `package.json` 과 `package-lock.json` 이 생성되고, 설치한 패키지는 `node_modules` 디렉터리에 저장됩니다.

우리는 Hardhat 구동 환경과 OpenZeppelin 라이브러리 사용을 위해 `package.json` 을 아래와 같이 수정해야 합니다.

```
{
  "name": "sub-pjt-1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "@nomicfoundation/hardhat-ethers": "^4.0.14",
    "@openzeppelin/contracts": "^5.6.1"
  },
  "devDependencies": {
    "@nomicfoundation/hardhat-toolbox-mocha-ethers": "^3.0.7",
    "hardhat": "^3.9.1"
  }
}
```

설치가 끝나면 Hardhat 설정에서 솔리디티 컴파일러 버전을 지정해야 합니다. 다음 명령어로 파일을 생성 합니다.

```
PS C:\Users\student\Ssafy\sub-pjt-1> New-Item hardhat.config.js
```

생성 후 해당 파일을 아래와 같이 편집합니다.

```
import { defineConfig } from "hardhat/config";
import hardhatToolboxMochaEthers from "@nomicfoundation/hardhat-toolbox-mocha-ethers";

export default defineConfig({
  plugins: [hardhatToolboxMochaEthers],
  solidity: "0.8.24",
});
```

컨트랙트와 테스트 코드를 작성한 뒤에는 프로젝트 최상위 디렉터리에서 다음 명령을 실행합니다.

```
PS C:\SSAFY\sub-pjt-1> npx hardhat compile
PS C:\SSAFY\sub-pjt-1> npx hardhat test
```

compile 명령은 솔리디티 컨트랙트를 컴파일하며, **test** 명령은 **test** 디렉터리에 작성된 테스트를 실행합니다. PowerShell 에서 스크립트 실행이 제한되었다는 오류가 발생하면 관리자 권한으로 전체 보안 설정을 해제하지 말고, 먼저 **npm.cmd** 또는 **npx.cmd** 명령을 사용합니다.

(5) 토큰의 수량과 최소 단위

ERC-20 토큰의 잔액은 정수로 저장됩니다. OpenZeppelin ERC-20 의 기본 decimals 값은 18 이므로 화면에서 보이는 1 토큰은 컨트랙트 내부에서 10^{18} 로 표현됩니다. 예를 들어 초기 공급량을 1,000,000 토큰으로 설정하려면 컨트랙트에 전달하는 실제 값은 다음과 같습니다.

$$1,000,000 * 10^{18}$$

테스트 코드와 배포 스크립트에서는 큰 정수를 직접 작성하지 않고 ethers 의 단위 변환 함수를 사용합니다.

```
const initialSupply = ethers.parseUnits("1000000", 18);
const betAmount = ethers.parseUnits("10", 18);
```

parseUnits 는 사람이 읽는 토큰 수량을 컨트랙트에서 사용하는 최소 단위로 변환합니다. 반대로 조회한 잔액을 화면에 표시할 때는 **formatUnits** 를 사용할 수 있습니다.

(6) TestToken 컨트랙트 작성

contracts/TestToken.sol 파일을 생성하고 다음 구조로 구현합니다.

생성자는 토큰 이름, 심벌, 초기 공급량을 전달받습니다. **ERC20(name_, symbol_)** 은 상위 ERC-20 컨트랙트의 생성자를 호출하여 토큰 정보를 설정합니다. 꼭 OpenZeppelin 에 방문하여 ERC-20 컨트랙트의 소스코드를 참고하기 바랍니다. **_mint(msg.sender, initialSupply_)** 는 초기 공급량을 생성하고 그 수량을 컨트랙트 배포자에게 지급합니다.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
```

```
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract TestToken is ERC20 {
    constructor(
        string memory name_,
        string memory symbol_,
        uint256 initialSupply_
    ) ERC20(name_, symbol_) {
        _mint(msg.sender, initialSupply_);
    }
}
```

OpenZeppelin 에서 상속받는 주요 함수는 다음과 같습니다.

함수	호출 주체	역할
<code>name()</code>	누구나	토큰 이름을 조회합니다.
<code>symbol()</code>	누구나	토큰 심벌을 조회합니다.
<code>decimals()</code>	누구나	토큰의 소수점 자릿수를 조회합니다.
<code>totalSupply()</code>	누구나	현재 발행된 전체 토큰 수량을 조회합니다.
<code>balanceOf(account)</code>	누구나	특정 주소의 잔액을 조회합니다.
<code>transfer(to, amount)</code>	토큰 보유자	자신의 토큰을 다른 주소로 전송합니다.
<code>approve(spender, amount)</code>	토큰 보유자	다른 주소에 토큰 사용 권한을 부여합니다.
<code>allowance(owner, spender)</code>	누구나	위임된 토큰 사용 한도를 조회합니다.
<code>transferFrom(from, to, amount)</code>	사용 권한을 받은 주소	허용된 범위에서 다른 주소의 토큰을 전송합니다.

직접 작성한 함수는 생성자뿐이지만, ERC-20 을 상속했기 때문에 표준 토큰 기능을 모두 사용할 수 있습니다. 컨트랙트를 컴파일한 뒤 ABI 를 확인하여 위 함수들이 포함되어 있는지 확인합니다.

컨트랙트를 모두 작성하였다면, 다음 명령어를 통해 컴파일을 진행하세요.

```
PS C:\SSAFY\sub-pjt-1> npx hardhat compile
```

(7) 토큰 배포 및 사용자 지급

`scripts/deploy-test-token.js` 파일을 생성하고 다음 구조로 구현합니다.

테스트 토큰을 배포할 때는 토큰 이름, 심벌, 최소 단위로 변환한 초기 공급량을 생성자에 전달합니다. 다음 예시는 SSAFY Test Token, STT, 1,000,000 토큰을 생성합니다.

```
import { network } from "hardhat";

async function main() {
  const { ethers } = await network.create();
  const initialSupply = ethers.parseUnits("1000000", 18);
  const token = await ethers.deployContract("TestToken", [
    "SSAFY Test Token",
    "STT",
    initialSupply,
  ]);

  await token.waitForDeployment();
  console.log("TestToken:", await token.getAddress());
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

작성이 완료되면 다음 명령어를 통해 토큰을 배포합니다. 잘 배포가 되었다면 토큰 컨트랙트의 주소를 리턴받게 됩니다.

```
PS C:\SSAFY\sub-pjt-1> npx hardhat run .\scripts\deploy-test-token.js
TestToken: {Token Contract Address}
```

배포가 완료되면 배포자 주소가 전체 초기 공급량을 보유합니다. 예측 참여 테스트에 사용할 계정에는 `transfer` 를 호출하여 토큰을 지급합니다.

```
const userAmount = ethers.parseUnits("100", 18);

await token.transfer(user1.address, userAmount);
await token.transfer(user2.address, userAmount);
```

전송 이후에는 배포자의 잔액이 지급한 수량만큼 감소하고 각 사용자의 잔액이 증가해야 합니다. 트랜잭션이 성공했다는 사실만 확인하지 말고, 전송 전후의 `balanceOf` 값을 비교하여 실제 상태가 변경되었는지 확인합니다.

```
const balance1 = await token.balanceOf(user1.address);
const balance2 = await token.balanceOf(user2.address);
```

(8) `approve` 와 `transferFrom` 실습

다음 실습에서 구현할 예측 컨트랙트는 사용자의 토큰을 직접 보관해야 합니다. 그러나 예측 컨트랙트는 사용자의 개인키를 가지고 있지 않으므로 사용자의 토큰을 임의로 이동할 수 없습니다. 사용자는 먼저 `approve` 를 호출하여 예측 컨트랙트 또는 실습용 대리 계정에 사용 한도를 부여해야 합니다.

이번 단계에서는 예측 컨트랙트 대신 `spender` 계정을 사용하여 승인과 대리 전송 흐름을 확인합니다.

```
const approvedAmount = ethers.parseUnits("30", 18);

await token.connect(user1).approve(spender.address, approvedAmount);
const allowance = await token.allowance(user1.address, spender.address);
console.log(ethers.formatUnits(allowance, 18));
```

승인받은 `spender` 는 허용 수량 안에서 `transferFrom` 을 호출할 수 있습니다.

```
const transferAmount = ethers.parseUnits("10", 18);

await token
  .connect(spender)
  .transferFrom(user1.address, receiver.address, transferAmount);
```

대리 전송이 완료되면 다음 상태가 모두 변경되어야 합니다.

- ① `user1` 의 잔액이 10 토큰 감소합니다.
- ② `receiver` 의 잔액이 10 토큰 증가합니다.
- ③ `spender` 에게 부여된 허용 수량이 30 토큰에서 20 토큰으로 감소합니다.

`approve` 는 토큰을 즉시 전송하지 않습니다. 토큰 사용 권한과 최대 수량만 기록하며, 실제 토큰 이동은 `transferFrom` 이 실행될 때 발생합니다. 이 차이를 이해해야 다음 과업에서 예측 참여 트랜잭션을 올바르게 구현할 수 있습니다.

(9) 테스트 코드 작성

테스트에서는 컨트랙트가 정상적으로 동작하는 경우뿐 아니라 잘못된 요청을 거부하는지도 확인해야 합니다. 각 테스트는 독립적으로 실행될 수 있도록 테스트 시작 전에 새로운 토큰 컨트랙트를 배포합니다.

필수 테스트 시나리오는 다음과 같습니다.

구분	테스트 내용	기대 결과
배포	토큰 이름과 심벌을 조회합니다.	생성자에 입력한 값과 일치합니다.
초기 공급	전체 공급량과 배포자 잔액을 조회합니다.	두 값이 초기 공급량과 일치합니다.
일반 전송	배포자가 사용자에게 토큰을 전송합니다.	송신자 잔액은 감소하고 수신자 잔액은 증가합니다.
잔액 초과 전송	보유 잔액보다 많은 토큰을 전송합니다.	트랜잭션이 실패합니다.
사용 승인	사용자가 spender 에게 사용 한도를 부여합니다.	allowance 가 승인 수량과 일치합니다.
대리 전송	spender 가 승인 수량 안에서 토큰을 전송합니다.	잔액과 남은 허용 수량이 변경됩니다.
미승인 대리 전송	승인 없이 transferFrom 을 호출합니다.	트랜잭션이 실패합니다.
승인 수량 초과	허용된 수량보다 많이 대리 전송합니다.	트랜잭션이 실패합니다.

다음은 예시 테스트 코드입니다. 이 코드를 참고하여 조건을 만족하는 테스트 코드를 직접 작성해보세요.

```
import { expect } from "chai";
import { network } from "hardhat";

const { ethers } = await network.create();

describe("TestToken", function () {
  async function deployToken() {
    const [deployer, user1, spender, receiver] = await ethers.getSigners();
```

```

const initialSupply = ethers.parseUnits("1000000", 18);
const token = await ethers.deployContract("TestToken", [
  "SSAFY Test Token",
  "STT",
  initialSupply,
]);
return { token, deployer, user1, spender, receiver, initialSupply };
}

it("mints the initial supply to the deployer", async function () {
  const { token, deployer, initialSupply } = await deployToken();
  expect(await token.totalSupply()).to.equal(initialSupply);
  expect(await token.balanceOf(deployer.address)).to.equal(initialSupply);
});

it("allows an approved spender to transfer tokens", async function () {
  const { token, deployer, user1, spender, receiver } = await deployToken();
  const userBalance = ethers.parseUnits("100", 18);
  const approvedAmount = ethers.parseUnits("30", 18);
  const transferAmount = ethers.parseUnits("10", 18);
  await token.connect(deployer).transfer(user1.address, userBalance);
  await token.connect(user1).approve(spender.address, approvedAmount);
  await token
    .connect(spender)
    .transferFrom(user1.address, receiver.address, transferAmount);

  expect(await token.balanceOf(receiver.address)).to.equal(transferAmount);
  expect(await token.allowance(user1.address, spender.address)).to.equal(
    approvedAmount - transferAmount,
  );
});
});

```

제시된 코드는 기본 구조를 보여주기 위한 예시입니다. 나머지 필수 시나리오는 직접 테스트로 추가해야 합니다. 실패 테스트에서는 단순히 오류가 발생했다는 사실뿐 아니라, 사용 중인 OpenZeppelin 버전에서 반환하는 **custom error** 또는 **revert** 조건이 예상과 일치하는지도 확인합니다.

테스트 코드 작성이 완료되었다면, 이를 아래 명령어를 통해 테스트를 수행합니다.

```
PS C:\SSAFY\sub-pjt-1> npx hardhat test .\test\TestToken.test.js
```

2) 과제 2: 예측 컨트랙트 구현

과제 1에서는 예측 참여에 사용할 테스트 ERC-20 토큰을 구현하고, **approve**와 **transferFrom**을 이용한 대리 전송 방식을 확인했습니다. 이번 과제에서는 해당 토큰을 실제로 사용하는 단일 예측 컨트랙트를 구현합니다.

사용자는 여러 선택지 중 하나를 선택하고 일정 수량의 토큰을 전송합니다. 예측 컨트랙트는 사용자가 미리 승인한 토큰을 전달받고, 사용자별 참여 정보와 선택지별 참여 금액을 블록체인에 기록해야 합니다. 같은 사용자의 중복 참여, 존재하지 않는 선택지 선택, 0토큰 참여와 같은 잘못된 요청은 컨트랙트에서 직접 차단해야 합니다.

이번 과제에서는 하나의 컨트랙트가 하나의 예측 이벤트만 표현합니다. 여러 이벤트를 생성하는 팩토리 구조와 결과 확정, 정산 금액 계산, 실제 보상 전송은 아직 구현하지 않습니다. 예측 참여에 필요한 상태와 토큰 이동을 정확히 구현하는 것이 이번 과제의 목표입니다.

(1) 학습 목표

이번 과제를 완료하면 다음 내용을 설명하고 구현할 수 있어야 합니다.

구분	학습 목표
컨트랙트 연결	생성자에서 ERC-20 토큰 주소를 전달받아 다른 컨트랙트와 연결할 수 있습니다.
선택지 관리	예측 선택지를 배열에 저장하고 조회할 수 있습니다.
참여 정보 설계	struct 와 mapping 을 사용하여 사용자별 참여 정보를 저장할 수 있습니다.
토큰 수령	safeTransferFrom 으로 승인된 토큰을 예측 컨트랙트에 전달할 수 있습니다.
입력값 검증	예측 참여 금액, 선택지 번호, 중복 참여 여부를 검증할 수 있습니다.
금액 집계	선택지별 참여 금액과 전체 참여 금액을 일관되게 집계할 수 있습니다.
이벤트 기록	예측 참여 결과를 솔리디티 이벤트로 기록할 수 있습니다.
컨트랙트 테스트	토큰 잔액과 예측 상태가 함께 변경되는지 테스트할 수 있습니다.

(2) 구현 요구사항

PredictionEvent 컨트랙트는 하나의 예측 이벤트를 표현합니다. 컨트랙트 배포 시 사용할 ERC-20 토큰 주소와 예측 선택지 목록을 전달받고, 배포 이후 사용자의 예측 참여를 처리합니다.

번호	요구사항
1	컨트랙트 이름은 PredictionEvent 로 작성합니다.
2	생성자에서 테스트 ERC-20 토큰의 컨트랙트 주소를 전달받습니다.
3	생성자에서 최소 2 개의 예측 선택지를 전달받아 저장합니다.
4	사용자는 유효한 선택지 하나와 0 보다 큰 참여 금액을 입력할 수 있습니다.
5	사용자가 예측 컨트랙트에 충분한 사용 한도를 승인한 경우에만 참여할 수 있습니다.
6	예측 참여 시 사용자의 토큰이 예측 컨트랙트 주소로 이동해야 합니다.
7	사용자별 선택지, 참여 금액, 참여 여부를 저장합니다.
8	같은 사용자는 해당 예측에 한 번만 참여할 수 있습니다.
9	선택지별 총 참여 금액과 전체 참여 금액을 저장합니다.
10	참여 완료 시 사용자, 선택지, 참여 금액을 포함한 이벤트를 발생시킵니다.
11	선택지 목록과 사용자 참여 정보를 조회할 수 있어야 합니다.
12	정상 및 실패 시나리오를 자동화된 테스트 코드로 검증합니다.

선택지는 배열의 인덱스를 식별자로 사용합니다. 첫 번째 선택지는 0, 두 번째 선택지는 1 입니다. 선택지 이름의 길이나 최대 선택지 개수는 이번 과제에서 별도로 제한하지 않습니다.

(3) 실습 준비

기존 프로젝트에 **PredictionEvent.sol** 과 테스트 파일을 추가하면 다음과 같은 구조가 됩니다.

sub-pjt-1/

```
├── contracts/
│   ├── TestToken.sol
│   └── PredictionEvent.sol.
├── test/
│   ├── TestToken.test.js
│   └── PredictionEvent.test.js
├── scripts/
│   ├── deploy-test-token.js
│   └── deploy-prediction-event.js
├── hardhat.config.js
└── package.json
```

(4) 컨트랙트 설계

예측 참여를 구현하기 전에 어떤 상태를 블록체인에 저장할지 정의해야 합니다. 하나의 참여가 성공하면 사용자 정보, 선택지별 합계, 전체 합계가 함께 변경되어야 합니다.

필요한 상태는 다음과 같습니다.

상태	자료형	역할
bettingToken	IERC20	예측 참여에 사용하는 ERC-20 토큰을 가리킵니다.
options	string[]	사용자가 선택할 수 있는 예측 선택지 목록입니다.
bets	mapping (address => Bet)	주소별 참여 정보를 저장합니다.
optionTotalBets	mapping (uint256 => uint256)	선택지별 전체 참여 금액을 저장합니다.
totalBetAmount	uint256	모든 선택지에 모인 참여 금액의 합계입니다.

사용자별 참여 정보는 **Bet** 구조체로 표현합니다.

```
struct Bet {
    uint256 optionId;
    uint256 amount;
    bool participated;
}
```

participated 값은 중복 참여 여부를 판단하는 데 사용합니다. 참여 금액이 0 보다 커야 한다는 조건만으로 참여 여부를 판단할 수도 있지만, 참여 여부를 별도 필드로 표현하면 상태의 의미가 명확해지고 이후 로직을 확장하기 쉽습니다.

사용자 주소가 **bets mapping**의 키가 되고, **Bet** 구조체의 선택지와 금액으로 연결되는 모습과 각 선택지의 **optionTotalBets**가 증가하는 관계를 함께 표현하는 구성을 권장합니다.

(5) 컨트랙트 작성

Contracts\PredictionEvent.sol 파일을 생성하고 다음과 같이 작성합니다.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

contract PredictionEvent {
    using SafeERC20 for IERC20;

    struct Bet {
        uint256 optionId;
        uint256 amount;
        bool participated;
    }

    IERC20 public immutable bettingToken;
    string[] private options;

    mapping(address => Bet) private bets;
    mapping(uint256 => uint256) public optionTotalBets;
    uint256 public totalBetAmount;

    event BetPlaced(
        address indexed user,
        uint256 indexed optionId,
        uint256 amount
    );

    constructor(address tokenAddress_, string[] memory options_) {
        require(tokenAddress_ != address(0), "Invalid token address");
        require(options_.length >= 2, "At least two options required");

        bettingToken = IERC20(tokenAddress_);
        options = options_;
    }

    function placeBet(uint256 optionId_, uint256 amount_) external {
        require(optionId_ < options.length, "Invalid option");
        require(amount_ > 0, "Amount must be greater than zero");
```

```

require(!bets[msg.sender].participated, "Already participated");

bets[msg.sender] = Bet({
    optionId: optionId_,
    amount: amount_,
    participated: true
});
optionTotalBets[optionId_] += amount_;
totalBetAmount += amount_;

bettingToken.safeTransferFrom(msg.sender, address(this), amount_);

emit BetPlaced(msg.sender, optionId_, amount_);
}

function getOptions() external view returns (string[] memory) {
    return options;
}

function getOptionCount() external view returns (uint256) {
    return options.length;
}

function getBet(address user_) external view returns (Bet memory) {
    return bets[user_];
}
}

```

IERC20은 외부 ERC-20 토큰과 통신하기 위한 표준 인터페이스입니다. 예측 컨트랙트는 토큰을 직접 발행하지 않고, 생성자에서 전달받은 토큰 컨트랙트의 함수를 호출합니다. **immutable**로 선언한 **bettingToken**은 생성 시 한 번 설정된 뒤 변경되지 않습니다.

SafeERC20은 ERC-20 토큰 전송을 안전하게 처리하기 위한 OpenZeppelin 라이브러리입니다. **safeTransferFrom**은 토큰 전송이 실패하거나 비표준 반환값을 사용하는 경우를 안전하게 처리합니다. 이번 실습에서는 첫 번째 과제에서 만든 표준 토큰을 사용하지만, 다른 ERC-20 토큰과 연결할 수 있는 구조를 학습하기 위해 **SafeERC20**을 적용합니다.

(6) 생성자 검증

예측 컨트랙트는 배포 시 토큰 주소와 선택지를 전달받습니다. 잘못된 값으로 배포된 컨트랙트는 이후 수정하기 어려우므로 생성자에서 필수 조건을 검증합니다.

```

require(tokenAddress_ != address(0), "Invalid token address");
require(options_.length >= 2, "At least two options required");

```

영 주소가 토큰 주소로 설정되면 ERC-20 함수를 호출할 수 없습니다. 또한 예측은 비교할 대상이 필요하므로 최소 2 개의 선택지를 요구합니다.

배포 스크립트에서는 먼저 **TestToken** 을 배포한 뒤 해당 주소를 **PredictionEvent** 생성자에 전달합니다.

```
const { ethers } = require("hardhat");

async function main() {
  const initialSupply = ethers.parseUnits("1000000", 18);
  const TestToken = await ethers.getContractFactory("TestToken");
  const token = await TestToken.deploy(
    "SSAFY Test Token",
    "STT",
    initialSupply,
  );
  await token.waitForDeployment();

  const PredictionEvent = await ethers.getContractFactory("PredictionEvent");
  const prediction = await PredictionEvent.deploy(
    await token.getAddress(),
    ["선택지 A", "선택지 B"],
  );
  await prediction.waitForDeployment();

  console.log("TestToken:", await token.getAddress()); console.log("PredictionEvent:", await prediction.getAddress());
}

main().catch((error) => { console.error(error); process.exitCode = 1; });
```

로컬 Hardhat 네트워크에 배포 스크립트를 실행하려면 다음 명령을 사용합니다.

```
C:\SSAFY\sub-pjt-1> npx hardhat run .\scripts\deploy-prediction-event.js
```

(7) 예측 참여 처리

사용자가 예측에 참여하려면 두 번의 트랜잭션이 필요합니다.

- ① 사용자가 `TestToken.approve` 를 호출하여 예측 컨트랙트에 토큰 사용 권한을 부여합니다.
- ② 사용자가 `PredictionEvent.placeBet` 을 호출하여 선택지와 참여 금액을 제출합니다.

`placeBet` 은 다음 순서로 동작합니다.

순서	처리 내용
1	선택지 번호가 배열 범위 안에 있는지 확인합니다.
2	참여 금액이 0 보다 큰지 확인합니다.
3	호출자가 아직 참여하지 않았는지 확인합니다.
4	사용자 참여 정보를 저장합니다.
5	선택지별 금액과 전체 금액을 증가시킵니다.
6	사용자의 토큰을 예측 컨트랙트 주소로 이동합니다.
7	<code>BetPlaced</code> 이벤트를 발생시킵니다.

상태를 먼저 변경하고 외부 토큰 컨트랙트를 호출하는 방식은 Checks-Effects-Interactions 패턴을 따릅니다. `safeTransferFrom` 이 실패하면 트랜잭션 전체가 되돌려지므로 앞에서 변경한 참여 정보와 집계 금액도 저장되지 않습니다. 따라서 토큰은 이동하지 않았는데 참여 정보만 남는 상태가 발생하지 않습니다. 사용자, ERC-20 토큰 컨트랙트, 예측 컨트랙트를 나란히 배치하고 `approve`, `placeBet`, `safeTransferFrom`, 상태 저장, `BetPlaced` 이벤트의 실행 순서를 표현하는 구성을 권장합니다.

(8) 예측 정보 조회

블록체인에 저장된 예측 정보를 확인할 수 있도록 조회 함수를 제공합니다.

조회 함수	반환 내용
<code>getOptions()</code>	전체 선택지 <code>label</code> 을 반환합니다.

<code>getOptionCount()</code>	선택지 개수를 반환합니다.
<code>getBet(user)</code>	해당 주소의 선택지, 참여 금액, 참여 여부를 반환합니다.
<code>optionTotalBets(optionId)</code>	특정 선택지에 모인 전체 금액을 반환합니다.
<code>totalBetAmount()</code>	모든 선택지에 모인 전체 금액을 반환합니다.
<code>bettingToken()</code>	예측에 사용하는 ERC-20 토큰 주소를 반환합니다.

조회 함수는 상태를 변경하지 않으므로 **view**로 선언합니다. **public** 상태 변수에는 솔리디티가 자동으로 **getter** 함수를 생성하므로 별도의 조회 함수를 중복해서 만들 필요가 없습니다.

사용자별 참여 정보를 저장하는 **bets mapping**은 **private**으로 선언하고 **getBet**을 통해 조회합니다. 이를 통해 반환 구조를 명확하게 유지하고 이후 참여 정보가 확장될 때 조회 함수도 함께 관리할 수 있습니다. **private**은 블록체인 데이터를 암호화하거나 숨기는 기능이 아니라 솔리디티 코드에서의 직접 접근 범위를 제한하는 기능입니다.

(9) 테스트 코드 작성

이번 과제의 테스트는 ERC-20 토큰과 예측 컨트랙트를 함께 배포한 뒤 진행합니다. 각 테스트에서 배포자, 사용자 2명, 승인받지 않은 사용자를 구분하여 사용합니다.

필수 테스트 시나리오는 다음과 같습니다.

구분	테스트 내용	기대 결과
배포	토큰 주소와 선택지 목록을 조회합니다.	생성자 입력값과 일치합니다.
선택지 부족	선택지를 1개만 전달하여 배포합니다.	배포가 실패합니다.
토큰 지급	테스트 사용자에게 토큰을 전송합니다.	사용자 잔액이 증가합니다.
예측 참여	승인 후 유효한 선택지에 참여합니다.	사용자 참여 정보가 저장됩니다.

이벤트 확인	정상적으로 예측에 참여합니다.	BetPlaced 이벤트가 정확한 값으로 발생 합니다.
토큰 이동	예측 참여 전후의 토큰 잔액을 비교 합니다.	사용자 잔액은 감소하고 컨트랙트 잔액은 증가합니다.
금액 집계	여러 사용자가 서로 다른 선택지에 참여합니다.	선택지별 합계와 전체 합계가 일치합니다.
중복 참여	같은 사용자가 두 번째 참여를 시도 합니다.	트랜잭션이 실패합니다.
0 토큰 참여	참여 금액으로 0 을 입력합니다.	트랜잭션이 실패합니다.
잘못된 선택지	존재하지 않는 선택지 번호를 입력합니다.	트랜잭션이 실패합니다.
승인 없는 참여	approve 없이 참여합니다.	토큰 전송과 참여가 모두 실패합니다.
승인 수량 초과	승인한 수량보다 많이 참여합니다.	토큰 전송과 참여가 모두 실패합니다.

과제 1 에서 작성했던 테스트 코드를 참고하여 조건에 맞는 테스트코드를 직접 작성해봅시다. 아래 화면은 테스트 코드를 돌리고 난 결과의 예시입니다.

```
C:\SSAFY\sub-pjt-1> npx hardhat test test/PredictionEvent.test.js
```

PredictionEvent

- ✓ 배포: 토큰 주소와 선택지 목록을 조회하면 생성자 입력값과 일치해야 합니다 (233ms)
- ✓ 선택지 부족: 선택지를 1개만 전달하면 배포가 실패해야 합니다
- ✓ 토큰 지급: 테스트 사용자에게 토큰을 전송하면 사용자 잔액이 증가해야 합니다
- ✓ 예측 참여: 승인 후 유효한 선택지에 참여하면 사용자 참여 정보가 저장되어야 합니다
- ✓ 이벤트 확인: 정상적으로 예측에 참여하면 **BetPlaced** 이벤트가 정확한 값으로 발생해야 합니다
- ✓ 토큰 이동: 예측 참여 전후 사용자 잔액은 감소하고 컨트랙트 잔액은 증가해야 합니다
- ✓ 금액 집계: 여러 사용자가 서로 다른 선택지에 참여하면 선택지별 합계와 전체 합계가 일치해야 합니다
- ✓ 중복 참여: 같은 사용자가 두 번째 참여를 시도하면 트랜잭션이 실패해야 합니다
- ✓ 0 토큰 참여: 참여 금액으로 0 을 입력하면 트랜잭션이 실패해야 합니다
- ✓ 잘못된 선택지: 존재하지 않는 선택지 번호를 입력하면 트랜잭션이 실패해야 합니다
- ✓ 승인 없는 참여: **approve** 없이 참여하면 토큰 전송과 참여가 모두 실패해야 합니다
- ✓ 승인 수량 초과: 승인한 수량보다 많이 참여하면 토큰 전송과 참여가 모두 실패해야 합니다

3) 과제 3: 정산 컨트랙트 구현

두 번째 과제에서는 사용자가 ERC-20 토큰으로 예측에 참여하고, 사용자별 참여 정보와 선택지별 참여 금액을 저장하는 **PredictionEvent** 컨트랙트를 구현했습니다. 이번 과제에서는 해당 컨트랙트에 결과 확정과 정산 금액 계산 기능을 추가합니다.

예측 결과는 아무나 입력할 수 없어야 하며, 한 번 확정된 결과는 다시 변경할 수 없어야 합니다. 결과가 확정되면 새로운 예측 참여를 차단하고, 적중한 사용자가 받을 수 있는 정산 금액을 전체 참여 풀과 정답 선택지의 참여 비율을 기준으로 계산해야 합니다.

이번 과제에서 정산은 받을 수 있는 토큰 수량을 계산하는 단계를 의미합니다. 실제로 적중자에게 토큰을 전송하는 기능은 구현하지 않습니다. 결과 확정과 보상 전송을 분리하면 정산 공식과 권한 조건을 먼저 작은 범위에서 검증할 수 있습니다.

(1) 학습 목표

이번 과제를 완료하면 다음 내용을 설명하고 구현할 수 있어야 합니다.

구분	학습 목표
운영자 권한	Ownable 을 사용하여 결과 확정 권한을 배포자로 제한할 수 있습니다.
상태 전환	예측 참여 상태와 결과 확정 상태를 구분할 수 있습니다.
결과 확정	유효한 정답 선택지를 한 번만 저장할 수 있습니다.
참여 차단	결과 확정 이후 새로운 예측 참여를 거부할 수 있습니다.
적중 판정	사용자 선택지와 정답 선택지를 비교할 수 있습니다.
비례 정산	전체 참여 풀을 적중자의 참여 비율에 따라 계산할 수 있습니다.
정수 연산	솔리디티 나눗셈의 절삭 특성과 잔여 토큰 발생 원인을 설명할 수 있습니다.
테스트	권한, 상태 전환, 정산 공식과 실패 조건을 테스트할 수 있습니다.

(2) 구현 요구사항

이번 과제에서는 별도의 정산 컨트랙트를 새로 배포하지 않고, 두 번째 과제의 **PredictionEvent** 컨트랙트에 정산 기능을 확장합니다. 하나의 예측 이벤트가 참여 정보와 결과를 함께 보유해야 정산에 필요한 상태를 일관되게 참조할 수 있기 때문입니다.

번호	요구사항
1	PredictionEvent 가 OpenZeppelin 의 Ownable 을 상속하도록 변경합니다.
2	컨트랙트 배포자를 최초 운영자로 설정합니다.
3	결과 확정 여부와 정답 선택지를 상태로 저장합니다.
4	운영자만 결과를 확정할 수 있어야 합니다.
5	존재하지 않는 선택지를 정답으로 지정할 수 없어야 합니다.
6	전체 참여 금액이 0 인 경우 결과를 확정할 수 없어야 합니다.
7	참여 금액이 없는 선택지는 정답으로 지정할 수 없어야 합니다.
8	결과는 한 번만 확정할 수 있어야 합니다.
9	결과 확정 이후 새로운 예측 참여를 차단합니다.
10	미참여자와 오답 참여자의 정산 금액은 0 이어야 합니다.
11	적중자의 정산 금액을 비례 배분 공식으로 계산합니다.
12	결과 확정과 정산 계산의 정상 및 실패 시나리오를 테스트합니다.

이번 과제에서는 정답 선택지에 참여자가 없는 상황을 허용하지 않습니다. 정답 선택지의 총 참여 금액이 0 이면 비례 정산 공식의 분모가 0 이 되므로 결과 확정을 차단합니다. 무승부, 전원 환불, 추가 배당금, 수수료, 잔여 토큰 회수와 같은 정책은 이후 단계에서 확장할 수 있습니다.

(3) 실습 파일 준비

이번 과제에서는 기존 파일을 확장하고 정산 테스트 파일을 추가합니다.

```
sub-pjt-1/
└─ contracts/
```

```

├── TestToken.sol
├── PredictionEvent.sol
├── test/
│   ├── TestToken.test.js
│   ├── PredictionEvent.test.js
│   └── Settlement.test.js
├── scripts/
│   ├── deploy-test-token.js
│   └── deploy-prediction-event.js
├── hardhat.config.js
└── package.json

```

Owable 은 OpenZeppelin Contracts 에 포함되어 있습니다. 패키지 설치 여부를 확인합니다. 패키지가 없다면 다음 명령으로 설치합니다.

```
C:\SSAFY\sub-pjt-1> npm install @openzeppelin/contracts
```

(4) 정산 상태 설계

정산 기능을 위해 기존 컨트랙트에 다음 상태를 추가합니다.

상태	자료형	역할
resolved	bool	예측 결과가 확정되었는지 표시합니다.
winningOptionId	uint256	운영자가 확정한 정답 선택지 번호입니다.
owner()	address	결과 확정 권한을 가진 운영자 주소입니다.

owner() 는 **Owable** 에서 제공하는 조회 함수입니다. 컨트랙트를 배포할 때 **Owable(msg.sender)** 를 호출하면 배포자 주소가 최초 운영자로 저장됩니다.

winningOptionId 는 결과가 확정되기 전에도 기본값 0 을 가집니다. 따라서 정답 선택지만 조회하여 결과 확정 여부를 판단하면 안 됩니다. 반드시 **resolved** 를 함께 확인해야 합니다.

결과 확정 시 다음 이벤트를 발생시킵니다.

```

event ResultResolved(uint256 indexed winningOptionId)

async function main() {
  const initialSupply = ethers.parseUnits("1000000", 18);
  const TestToken = await ethers.getContractFactory("TestToken");
  const token = await TestToken.deploy(
    "SSAFY Test Token",

```

```
"STT",
  initialSupply,
);
await token.waitForDeployment(); c

const PredictionEvent = await ethers.getContractFactory("PredictionEvent");
const prediction = await PredictionEvent.deploy(
  await token.getAddress(),
  ["선택지 A", "선택지 B"],
);
await prediction.waitForDeployment();

console.log("TestToken:", await token.getAddress()); console.log("PredictionEvent:", await
prediction.getAddress());
}

main().catch((error) => { console.error(error); process.exitCode = 1; });
```

이벤트를 사용하면 외부 애플리케이션이 결과 확정 트랜잭션을 찾고 어떤 선택지가 정
답으로 결정되었는지 확인할 수 있습니다. 참여 가능 상태에서는 `placeBet` 을 호출할 수
있고, 운영자가 `resolve` 를 호출하면 결과 확정 상태로 이동하여 이후 참여가 차단되는
흐름을 표현하는 구성을 권장합니다.

(5) 정산 공식

이번 과제에서는 전체 참여 풀을 정답 선택지에 참여한 사용자들이 자신의 참여 비율에
따라 나누어 갖는 방식을 사용합니다.

정산금액 = 전체 참여 금액 x 사용자 참여 금액 / 정답 선택지 총 참여 금액

$$\text{settlement amount} = \text{total amount} \times \frac{\text{user betting amount}}{\text{winning amount}}$$

예를 들어 전체 참여 금액이 100 토큰이고 정답 선택지에 모인 금액이 40 토큰이라고 가
정합니다. 정답 선택지에 사용자 A가 10 토큰, 사용자 B가 30 토큰을 참여했다면 정산
금액은 다음과 같습니다.

사용자	참여 금액	계산	정산 금액
사용자 A	10	$100 \times 10 \div 40$	25 토큰
사용자 B	30	$100 \times 30 \div 40$	75 토큰
오답 참여자	60	적중하지 않음	

사용자가 자신이 참여한 원금을 포함하여 전체 풀을 비례 배분받는 구조입니다. 위 예시에서 적중자들의 정산 금액 합계는 전체 참여 금액인 100 토큰과 일치합니다.

솔리디티는 정수 나눗셈에서 소수점 이하를 버립니다. 정산 결과가 나누어 떨어지지 않으면 컨트랙트에 일부 잔여 토큰이 남을 수 있습니다. 이번 과제에서는 절삭 결과를 그대로 사용하며, 잔여 토큰 전송 정책은 구현하지 않습니다.

(6) PredictEvent 컨트랙트 확장

contracts\PredictionEvent.sol 을 다음과 같이 확장합니다.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract PredictionEvent is Ownable {
    using SafeERC20 for IERC20;

    struct Bet {
        uint256 optionId;
        uint256 amount;
        bool participated;
    }

    IERC20 public immutable bettingToken;
    string[] private options;

    mapping(address => Bet) private bets;
    mapping(uint256 => uint256) public optionTotalBets;
    uint256 public totalBetAmount;

    bool public resolved;
    uint256 public winningOptionId;
    event BetPlaced(
        address indexed user,
        uint256 indexed optionId,
        uint256 amount
    );
    event ResultResolved(uint256 indexed winningOptionId);

    constructor(
        address tokenAddress_,
        string[] memory options_
    ) Ownable(msg.sender) {
        require(tokenAddress_ != address(0), "Invalid token address");
        require(options_.length >= 2, "At least two options required");

        bettingToken = IERC20(tokenAddress_);
    }
}
```

```

    options = options_;
}

function placeBet(uint256 optionId_, uint256 amount_) external {
    require(!resolved, "Prediction already resolved");
    require(optionId_ < options.length, "Invalid option");
    require(amount_ > 0, "Amount must be greater than zero");
    require(!bets[msg.sender].participated, "Already participated");

    bets[msg.sender] = Bet({
        optionId: optionId_,
        amount: amount_,
        participated: true
    });
    optionTotalBets[optionId_] += amount_;
    totalBetAmount += amount_;

    bettingToken.safeTransferFrom(msg.sender, address(this), amount_);

    emit BetPlaced(msg.sender, optionId_, amount_);
}

function resolve(uint256 winningOptionId_) external onlyOwner {
    require(!resolved, "Prediction already resolved");
    require(winningOptionId_ < options.length, "Invalid option");
    require(totalBetAmount > 0, "No bets placed");
    require(
        optionTotalBets[winningOptionId_] > 0,
        "No bets on winning option"
    );

    resolved = true;
    winningOptionId = winningOptionId_;

    emit ResultResolved(winningOptionId_);
}

function calculatePayout(address user_) external view returns (uint256) {
    require(resolved, "Prediction not resolved");

    Bet memory userBet = bets[user_];

    if (!userBet.participated || userBet.optionId != winningOptionId) {
        return 0;
    }

    uint256 winningPool = optionTotalBets[winningOptionId];
    return (totalBetAmount * userBet.amount) / winningPool;
}

function getOptions() external view returns (string[] memory) {
    return options;
}

function getOptionCount() external view returns (uint256) {
    return options.length;
}

```

```

    }

    function getBet(address user_) external view returns (Bet memory) {
        return bets[user_];
    }
}

```

OpenZeppelin Contracts 5.x의 **Ownable** 생성자는 최초 운영자 주소를 인자로 받습니다. **Ownable(msg.sender)**를 통해 **PredictionEvent** 배포자를 운영자로 지정합니다. 사용하는 OpenZeppelin 버전이 다르면 생성자 형식도 다를 수 있으므로 설치된 버전의 API를 확인합니다.

두 번째 과제인 **placeBet**에는 **require(!resolved, ...)** 조건이 추가되었습니다. 결과가 확정된 뒤에는 토큰 승인 여부와 관계없이 새로운 참여가 실패해야 합니다.

(7) 결과 확정 처리

resolve 함수는 운영자가 정답 선택지를 저장하고 예측 상태를 종료하는 함수입니다.

```
function resolve(uint256 winningOptionId_) external onlyOwner
```

결과 확정 조건은 다음과 같습니다.

조건	실패 이유
아직 결과가 확정되지 않았을 것	이미 확정된 결과의 변경을 막습니다.
정답 선택지가 존재할 것	배열 범위를 벗어난 선택지를 차단합니다.
전체 참여 금액이 0보다 클 것	참여가 없는 이벤트의 결과 확정을 막습니다.
정답 선택지의 참여 금액이 0보다 클 것	정산 계산 시 0으로 나누는 상황을 막습니다.

모든 조건을 통과하면 **resolved**를 **true**로 변경하고 **winningOptionId**를 저장합니다. 상태 저장 후 **ResultResolved** 이벤트를 발생시킵니다.

(8) 정산 금액 계산

`calculatePayout` 은 상태를 변경하지 않는 `view` 함수입니다. 따라서 함수를 호출해도 토큰이 전송되지 않으며, 계산 결과만 반환합니다.

함수는 다음 순서로 동작합니다.

- ① 결과가 확정되었는지 확인합니다.
- ② 조회 대상 사용자의 참여 정보를 가져옵니다.
- ③ 미참여자 또는 오답 참여자라면 0 을 반환합니다.
- ④ 정답 선택지의 전체 참여 금액을 조회합니다.
- ⑤ 비례 배분 공식으로 적중자의 정산 금액을 계산합니다.

(9) 테스트 코드 작성

정산 테스트에서는 여러 사용자가 양쪽 선택지에 서로 다른 금액을 참여한 상황을 준비합니다. 사용자 A와 B는 정답 선택지에 각각 10 토큰과 30 토큰을 참여하고, 사용자 C는 오답 선택지에 60 토큰을 참여합니다.

필수 테스트 시나리오는 다음과 같습니다.

구분	테스트 내용	기대 결과
운영자 확인	컨트랙트의 <code>owner</code> 를 조회합니다.	배포자 주소와 일치합니다.
결과 확정	운영자가 유효한 정답을 입력합니다.	결과 상태와 정답 선택지가 저장됩니다.
이벤트 확인	결과를 확정합니다.	<code>ResultResolved</code> 이벤트가 발생합니다.
권한 없는 확인	일반 사용자가 결과를 입력합니다.	트랜잭션이 실패합니다.
잘못된 선택 지	존재하지 않는 선택지를 정답으로 입력합니다.	트랜잭션이 실패합니다.
참여 없는 확인	참여 없이 결과를 입력합니다.	트랜잭션이 실패합니다.

정답 풀 없음	참여가 없는 선택지를 정답으로 입력합니다.	트랜잭션이 실패합니다.
중복 확정	이미 확정된 결과를 다시 입력합니다.	트랜잭션이 실패합니다.
확정 후 참여	결과 확정 이후 참여를 시도합니다.	트랜잭션이 실패합니다.
적중자 정산	적중자의 정산 금액을 조회합니다.	비례 배분 계산 결과와 일치합니다.
오답자 정산	오답 참여자의 정산 금액을 조회합니다.	0 을 반환합니다.
미참여자 정산	참여하지 않은 주소의 정산 금액을 조회합니다.	0 을 반환합니다.
확정 전 조회	결과 확정 전에 정산 금액을 조회합니다.	호출이 실패합니다.
토큰 잔액 유지	정산 금액만 조회합니다.	

아래 화면은 테스트 코드 실행 결과의 예시입니다.

```
C:\SSAFY\sub-pjt-1> npx hardhat test test/Settlement.test.js
```

PredictionEvent settlement

- ✓ 운영자 확인: 컨트랙트 owner 를 조회하면 배포자 주소와 일치해야 합니다 (246ms)
- ✓ 결과 확정: 운영자가 유효한 정답을 입력하면 결과 상태와 정답 선택지가 저장되어야 합니다
- ✓ 이벤트 확인: 결과를 확정하면 ResultResolved 이벤트가 발생해야 합니다
- ✓ 권한 없는 확정: 일반 사용자가 결과를 입력하면 트랜잭션이 실패해야 합니다
- ✓ 잘못된 선택지: 존재하지 않는 선택지를 정답으로 입력하면 트랜잭션이 실패해야 합니다
- ✓ 참여 없는 확정: 참여 없이 결과를 입력하면 트랜잭션이 실패해야 합니다
- ✓ 정답 풀 없음: 참여가 없는 선택지를 정답으로 입력하면 트랜잭션이 실패해야 합니다
- ✓ 중복 확정: 이미 확정된 결과를 다시 입력하면 트랜잭션이 실패해야 합니다
- ✓ 확정 후 참여: 결과 확정 이후 참여를 시도하면 트랜잭션이 실패해야 합니다
- ✓ 적중자 정산: 적중자의 정산 금액은 비례 배분 계산 결과와 일치해야 합니다
- ✓ 오답자 정산: 오답 참여자의 정산 금액을 조회하면 0 을 반환해야 합니다
- ✓ 미참여자 정산: 참여하지 않은 주소의 정산 금액을 조회하면 0 을 반환해야 합니다
- ✓ 확정 전 조회: 결과 확정 전에 정산 금액을 조회하면 호출이 실패해야 합니다
- ✓ 토큰 잔액 유지: 정산 금액만 조회하면 컨트랙트 토큰 잔액이 변하지 않아야 합니다

14 passing (408ms)

본 문서는 삼성 청년 SW·AI 아카데미의 콘텐츠 자산으로,
무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.